

classify different types of operating systems suitable for this scenario and justify your choice

For modern OS supporting multiple users and multiple processes with shared resource:

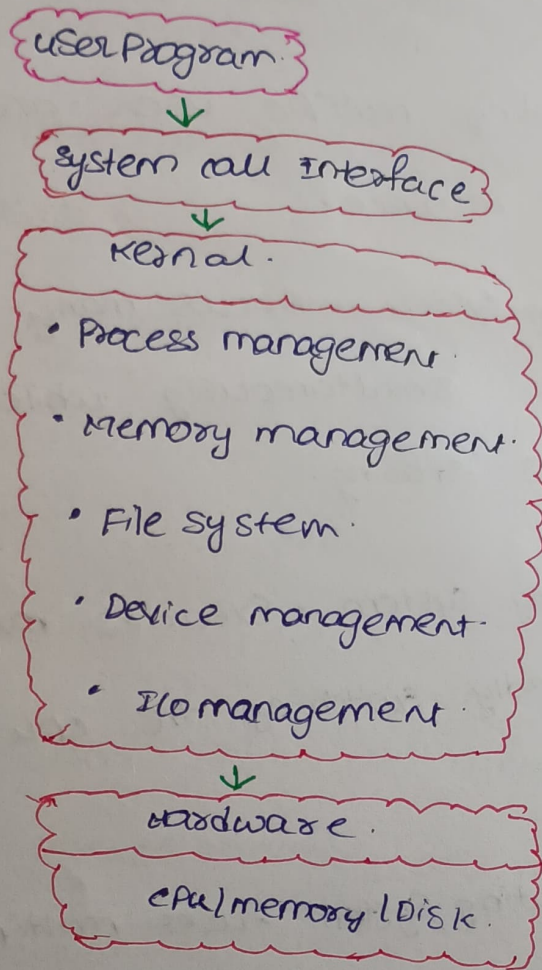
- Multi-user operating system - Allows many users to access the computer simultaneously while maintaining security and resource sharing.
- Multi tasking operating system - Executes multiple programs at the same time by rapidly switching the CPU among the process.
- Multiprocessing operating system - Uses multiple CPU cores to improve performance and reliability.
- Time sharing operating system - Allocates CPU time slices to each user/process for quick response

Justification:-

A multi user time sharing operating system is most suitable because it efficiently shares CPU, memory files and I/O devices among many users and process while ensuring fairness and security

b)

Draw the structure of an operating system.



c) Sequence of system calls to create and execute.

a) new process :-

```
pid = fork(); // creates
```

```
if (pid == 0)
```

```
{ execve("/bin/program", args, env); // executes
```

```
}
```

```
else
```

```
{ wait(NULL); // parent waits for child
```

```
}
```

```
exit(0); // terminate
```


3. consider a scenario where an operating system must manage process scheduling, memory allocation and device communication efficiently.

a. Explain the major functions of an OS in handling these tasks.

A,

1. Process Scheduling:-

- creates scheduling, and terminate processes.
- Allocates CPU efficiently using scheduling algorithms.

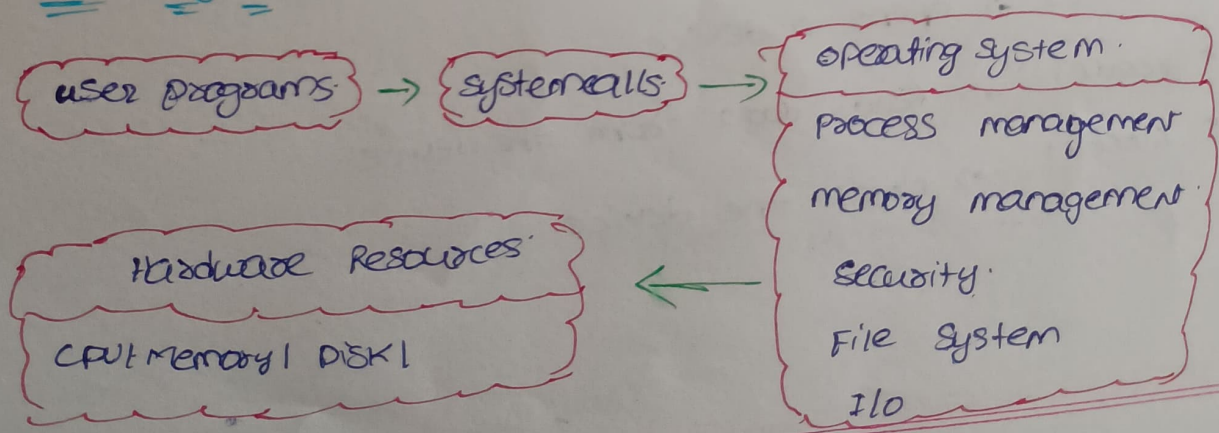
2. Memory Management:-

- Allocates and deallocates memory.
- Supports virtual memory and memory protection.

3. Device Management:-

- controls I/O devices using device drivers.
- Manages input/output operations and interrupts.

b. Block diagram:-



c) write system call examples for memory allocation and device management.

Memory management:-

```
void *ptr = mmap (Null, 4096,  
                  PROT_READ|PROT_WRITE,  
                  MAP_PRIVATE|MAP_ANONYMOUS,  
                  -1, 0);
```

```
munmap (ptr, 4096);
```

* mmap() → Allocates virtual memory

* munmap() → Releases allocated memory.

Device management:-

```
int fd = open ("dev/sda", O_RDONLY);
```

```
read (fd, buffer, 512);
```

```
close (fd);
```

* open() → opens a device.

* read() → Reads data from the device.

* close() → closes the device.